ASSIGNMENT-02

Question 1

A university has developed an Online Examination Management System in which students can register for examinations, select subjects, make payments, download admit cards, and view results. Develop a complete testing strategy for the system by explaining how testing fundamentals, testing types, static testing, dynamic testing, verification, and validation would be applied at different stages of development. Justify why each testing activity is required.

Answer

1.1 Introduction

The Online Examination Management System (OEMS) handles personal data, money (exam fees) and high-stakes information (admit cards and results). A defect here can cause a student to miss an exam, be charged twice or see a wrong result. Therefore a structured test strategy that starts at the requirements stage and continues until after release is required.

1.2 Testing Fundamentals Applied to OEMS

- **Testing shows presence of defects** - Application in OEMS: Test results are reported as "defects found", never as "system is bug free"; Why it is required: Gives management a realistic view of risk before the exam season.
- **Exhaustive testing is impossible** - Application in OEMS: Use Equivalence Partitioning and Boundary Value Analysis for fee amounts, dates, and subject counts instead of trying all inputs; Why it is required: Limits effort while keeping coverage high.
- **Early testing** - Application in OEMS: Requirements and design are reviewed before coding starts (e.g. "registration closes on date X" is checked for ambiguity); Why it is required: A defect found in requirements is far cheaper to fix than one found in production.
- **Defect clustering** - Application in OEMS: Payment, admit-card generation and result module receive more test effort; Why it is required: Most defects collect in a few complex modules.
- **Pesticide paradox** - Application in OEMS: Test cases are reviewed and updated every release (new exam pattern, new payment gateway); Why it is required: Same tests repeated again and again stop finding new defects.
- **Testing is context dependent** - Application in OEMS: Security and load testing get high priority because of personal data and registration-day traffic; Why it is required: A web application with payments needs different tests than an offline tool.
- **Absence-of-errors fallacy** - Application in OEMS: Usability testing and UAT with students and exam staff; Why it is required: A bug-free system that students cannot use is still a failure.

1.3 Verification and Validation

Verification answers "Are we building the product right?" and is done without running the code (reviews, walkthroughs, inspections). **Validation** answers "Are we building the right product?" and is done by running the software and checking it against user needs.

- **Requirement analysis** - Verification activity: Review SRS for ambiguity, e.g. "results should be shown quickly"; Validation activity: Prototype shown to exam cell staff and students.

- **System design** - Verification activity: Design and database schema reviews; check that payment and registration modules interface correctly; Validation activity: Check design against use-cases (register, pay, download admit card).
- **Coding** - Verification activity: Code reviews, static analysis tools, coding standard checks; Validation activity: Unit tests run by developers.
- **Testing** - Verification activity: Review of test plan, test cases and traceability matrix; Validation activity: Integration, system and acceptance testing.
- **Deployment / maintenance** - Verification activity: Release checklist and configuration review; Validation activity: Smoke test in production, regression testing for every patch.

1.4 Static Testing (without executing code)

- **Requirement review:** finds missing rules such as refund policy for failed payments or late registration.
- **Design inspection:** checks database constraints (one student, one exam, one seat) and security design (role based access).
- **Code review and static analysis:** finds unused variables, SQL injection risks and duplicate logic.
- **Test case review:** makes sure every requirement is covered by at least one test case.

Why required: static testing catches defects at the earliest stage when the cost of correction is lowest and it also improves understanding of the system among team members.

1.5 Dynamic Testing (executing the code)

- **Unit testing** - What is tested in OEMS: Fee calculation function, age/eligibility check, admit card PDF generator, grade calculation; Who / why: Developers; find logic errors early and cheaply.
- **Integration testing** - What is tested in OEMS: Registration to Payment Gateway to Admit Card; Result module to Student database; Who / why: Developers/testers; interfaces are the most common source of defects.
- **System testing** - What is tested in OEMS: Complete flows end to end: register, choose subjects, pay, download hall ticket, view result; Who / why: Independent testers; verifies the whole system against the SRS.
- **Acceptance testing (UAT)** - What is tested in OEMS: Real students and examination staff run real scenarios in a staging environment; Who / why: University; confirms the system fulfils business needs (validation).
- **Regression testing** - What is tested in OEMS: Re-running important tests after every fix or new feature; Who / why: Ensures a fix to payments does not break admit cards.

1.6 Testing Types Used

- **Functional** - Sample OEMS test: Student selects 5 subjects, pays, and gets confirmation e-mail; Justification: Confirms each feature works as specified.
- **Black-box techniques** - Sample OEMS test: EP/BVA for number of subjects (min 1, max 6), fee amount, date of birth; Justification: Gives maximum defect detection with fewer cases.
- **White-box** - Sample OEMS test: Statement and branch coverage of eligibility and fee calculation code; Justification: Makes sure all code paths are exercised.
- **Performance / load** - Sample OEMS test: 20,000 students registering at the last hour of registration; result day traffic; Justification: The system must not crash at peak time.
- **Security** - Sample OEMS test: SQL injection, session hijacking, access of another student's result by changing the URL (IDOR), payment tampering; Justification: Protects personal data and money.
- **Usability** - Sample OEMS test: Can a first time user complete registration without help?; Justification: Reduces help-desk load and user errors.

- **Compatibility** - Sample OEMS test: Chrome, Firefox, Edge, Android and iOS browsers; admit card prints correctly; Justification: Students use many devices.
- **Recovery / reliability** - Sample OEMS test: Payment deducted but gateway times out; server restart during registration; Justification: Prevents loss of money and data.
- **Regression & Smoke** - Sample OEMS test: Daily smoke test after every build; Justification: Quickly rejects a broken build.

1.7 Stage-wise Test Strategy Summary

- **Requirements** - Testing activities: Static review, testability analysis, acceptance criteria definition; Output: Reviewed SRS, defect log.
- **Design** - Testing activities: Design review, test planning, test design; Output: Test plan, test scenarios.
- **Implementation** - Testing activities: Code review, static analysis, unit testing; Output: Unit test report.
- **Integration** - Testing activities: Integration testing, API testing; Output: Integration test report.
- **System testing** - Testing activities: Functional, security, performance, usability, compatibility; Output: System test report, defect report.
- **Release** - Testing activities: UAT, smoke test, sign-off; Output: UAT sign-off.
- **Maintenance** - Testing activities: Regression testing, monitoring; Output: Regression suite results.

1.8 Conclusion

By combining static and dynamic testing, applying verification at every stage and validation with real users, the university can deliver a reliable, secure and usable OEMS. Each activity is justified by the risk it reduces: static testing reduces cost, dynamic testing finds run-time failures, and validation confirms that the system really serves students and the examination department.

Question 2

Consider an online banking application that allows users to log in, transfer money, add beneficiaries, pay bills, and view transaction history. Identify and analyze the possible defects that can be detected through static testing before executing the application. Explain how static testing can reduce the cost and effort of later dynamic testing.

Answer

Static testing is a verification technique in which the software and its related documents are examined without executing the program. For an online banking application it is especially important, because the system handles money and confidential data, and a defect that reaches production can cause financial loss, fraud and loss of customer trust. Static testing consists of reviews (informal review, walkthrough, technical review, inspection) and static analysis using tools that scan the source code. It can be applied to requirement documents, design documents, source code, test cases and even user manuals.

2.1 Meaning of Static Testing

Static testing examines work products (requirements, design documents, code, test cases) without executing the software. Techniques include reviews, walkthroughs, inspections and static analysis tools.

2.2 Defects Detectable by Static Testing

- **Requirements** - Possible defect: Ambiguity; Example in banking application: "Transfer should be processed quickly" (no time given); Impact if not found: Cannot be tested; different people interpret differently.
- **Requirements** - Possible defect: Missing requirement; Example in banking application: No daily transfer limit or no rule for failed transfer after debit; Impact if not found: Money loss, regulatory problems.
- **Requirements** - Possible defect: Contradiction; Example in banking application: One section says OTP is mandatory for every transfer, another says only for amount above Rs. 10,000; Impact if not found: Inconsistent behaviour and security gap.
- **Requirements** - Possible defect: Untestable statement; Example in banking application: "System should be user friendly."; Impact if not found: No pass/fail criteria.
- **Design** - Possible defect: Security design flaw; Example in banking application: Passwords stored in plain text; no session timeout designed; Impact if not found: Account takeover.
- **Design** - Possible defect: Interface mismatch; Example in banking application: Bill-pay module sends amount in paise while core banking expects rupees; Impact if not found: Wrong amounts deducted.
- **Design** - Possible defect: Missing error flow; Example in banking application: No design for transaction rollback when the beneficiary bank is down; Impact if not found: Money debited but not credited.
- **Code** - Possible defect: Hard-coded credentials / keys; Example in banking application: Database password inside source code; Impact if not found: Data breach.
- **Code** - Possible defect: Missing input validation / SQL injection; Example in banking application: Query built by string concatenation with username; Impact if not found: Unauthorised access to accounts.
- **Code** - Possible defect: Unused or uninitialised variables, dead code; Example in banking application: Variable "balance" used before assignment; Impact if not found: Unpredictable results.
- **Code** - Possible defect: Missing exception handling / resource leaks; Example in banking application: Database connection not closed after statement history query; Impact if not found: System slows down and crashes under load.
- **Code** - Possible defect: Duplicate and complex logic; Example in banking application: Same beneficiary validation copied in three places; Impact if not found: Fix applied in one place only.
- **Code** - Possible defect: Standards violation; Example in banking application: Inconsistent naming, no comments in transfer function; Impact if not found: Low maintainability.
- **Test documents** - Possible defect: Coverage gaps; Example in banking application: No test case for adding duplicate beneficiary; Impact if not found: Defects escape to production.

2.3 How Static Testing Reduces Cost and Effort of Dynamic Testing

- **Early defect removal:** a defect found in requirements may cost 1 unit to fix, in coding about 10 units, and in production 100 units or more (including financial loss and reputation damage).
- **Fewer defects reach dynamic testing:** testers spend time on real functional problems instead of blocked test cases caused by obvious errors.
- **Fewer re-test and regression cycles:** each defect found late requires fix, re-test, and regression of related modules.
- **Better test cases:** reviewed requirements are clear, so test design is faster and more accurate.
- **Finds defects dynamic testing can miss:** e.g. dead code, standard violations, security flaws in unexecuted paths, and missing requirements.
- **Shorter schedule:** less debugging time, lower rework cost and faster release.

- **Knowledge sharing:** reviewers learn the system and spread good practice.

2.4 Conclusion

For a financial application where one defect can mean real money loss, static testing is a cost-effective first line of defence. It improves the quality of inputs to dynamic testing and keeps later testing focused on behaviour, performance and integration instead of avoidable mistakes.

Question 3

A development team has submitted a module containing approximately 2,000 lines of code for review. The module follows inconsistent naming conventions, contains duplicated logic, has inadequate comments, and uses complex conditional statements. Design a formal review process for this module. Include review participants, preparation activities, review stages, defect classification, review checklist, documentation, and follow-up activities.

Answer

A formal review (inspection) is a structured, documented and rule-based way of examining a work product. It has defined roles, entry and exit criteria, checklists, recorded defects and follow-up. It is used when the work product is important or risky, or when the quality problems are many, as in this module. The aim is to find defects early, to enforce coding standards and to make the code easier to maintain. Since the given module has inconsistent naming, duplicated logic, poor comments and complex conditions, mostly maintainability defects are expected, but the complex conditional statements may also hide logical defects, so a formal inspection is the right choice.

3.1 Choice of Review Type

A formal inspection (Fagan-style) is selected because the module is large, has multiple quality problems and needs recorded, measurable results. Since 2,000 LOC is too large for a single session (ideal inspection rate is about 150-200 LOC per hour), the code will be divided into **4 parts of about 500 LOC**, each reviewed in a separate 2-hour session.

3.2 Review Participants and Roles

- **Moderator:** Experienced, neutral person (senior tester/QA). Plans the review, checks entry criteria, controls the meeting, ensures follow-up.
- **Author:** Developer of the module. Explains doubts, does not defend; fixes defects afterwards.
- **Reader:** Paraphrases the code line by line or block by block during the meeting.
- **Reviewers / Inspectors (2-3):** Senior developer, tester and a developer from a dependent module. Find defects individually before the meeting.
- **Recorder / Scribe:** Records every defect, its classification and decisions in the inspection log.
- **Manager (optional):** Does not attend the meeting to avoid pressure on author; receives metrics only.

3.3 Entry Criteria

- Code compiles cleanly and has no compiler warnings.
- Unit tests exist and pass.
- Static analysis tool already run and obvious violations fixed.
- Design document and coding standard are available to reviewers.

3.4 Review Stages

- **1. Planning** - Activity: Moderator selects team, splits module into 4 parts, schedules sessions, distributes material; Output: Review plan.
- **2. Kick-off / overview** - Activity: Author explains purpose, design and structure of the module (30 minutes). Checklist and roles explained; Output: Common understanding.
- **3. Individual preparation** - Activity: Each reviewer reads the code alone (about 2 hours per part) with the checklist and notes defects; Output: Individual defect lists.
- **4. Review meeting** - Activity: Reader goes through the code; reviewers raise defects; recorder logs them. No solutions are discussed in detail; maximum 2 hours; Output: Inspection log with classified defects.
- **5. Rework** - Activity: Author fixes all accepted defects, removes duplication, renames identifiers, simplifies conditions, adds comments; Output: Corrected code, response to each defect.
- **6. Follow-up** - Activity: Moderator verifies each fix. Decision: accept, accept with conditions, or re-inspect; Output: Closed log, sign-off.

3.5 Preparation Activities

- Reviewers read the coding standard and design specification.
- Run static analysis and cyclomatic complexity tools to highlight risky functions.
- Check the code against checklist and mark line numbers.
- Prepare questions about unclear logic.
- Submit defect list to moderator one day before the meeting.

3.6 Defect Classification

By severity:

- **Major / Critical** - Description: Wrong logic, security hole, crash; Example in this module: Complex nested condition gives wrong result for one case; unhandled exception.
- **Minor** - Description: Does not affect function but affects quality; Example in this module: Duplicated logic, long method, missing comments.
- **Cosmetic / Suggestion** - Description: Style only; Example in this module: Inconsistent naming (camelCase and snake_case mix), indentation.

By type: logic, interface, data handling, standards violation, documentation, performance, security, maintainability.

3.7 Review Checklist (for this module)

- Are naming conventions for variables, methods and classes consistent and meaningful?
- Is any logic duplicated that should be moved to a common function?
- Is every method documented with purpose, parameters and return value?
- Are complex if/else or nested conditions simplified (cyclomatic complexity below 10)?
- Are all inputs validated and all errors handled?
- Are resources (files, DB connections) closed?
- Are there magic numbers, dead code or unused variables?
- Does the code match the design and requirements?
- Are boundary conditions (null, empty, zero, maximum) handled?
- Is the code testable and covered by unit tests?

3.8 Documentation

- **Inspection log:** defect number, location (file/line), description, class, severity, reviewer, status.

- **Inspection summary report:** size reviewed, effort (person-hours), number of defects by severity, inspection rate, defects per KLOC.
- **Rework report** by author and **sign-off record** by moderator.

3.9 Follow-up Activities

- Moderator verifies that every major defect is fixed and re-checks the changes.
- If more than a threshold (e.g. 5 major defects per 500 LOC) are found, a re-inspection is scheduled.
- Metrics are stored for process improvement; common defects are added to the checklist and used for team training.
- Final decision recorded: Accepted / Accepted with modification / Re-inspect.

Question 4

Explain the difference between peer review and formal review using a realistic software project. Assume that five developers are working on a large E-Commerce Application. Design a peer-review procedure for one critical module and explain how defects should be identified, recorded, discussed, corrected, and re-verified.

Answer

Reviews are of different levels of formality. A peer review is a light-weight review in which one or two colleagues of the same level check the work of the author, usually through a pull request or a desk check. A formal review (inspection) is the most rigorous type and follows a defined process with a moderator, preparation, a meeting, a written log and follow-up. In a team of five developers, daily changes are checked by peer review because it is quick and cheap, while a critical module like payment is checked by formal review because the risk of a defect is high. Peer review is not a replacement for formal review but supports it.

4.1 Peer Review vs Formal Review

- **Formality** - Peer review: Informal or semi-formal; no strict process; Formal review (inspection): Documented, defined process with entry/exit criteria.
- **Participants** - Peer review: Colleagues of the author (equal level); Formal review (inspection): Moderator, author, reader, recorder, inspectors.
- **Preparation** - Peer review: Little or none; Formal review (inspection): Mandatory individual preparation with checklists.
- **Documentation** - Peer review: Comments in pull request / simple list; Formal review (inspection): Inspection log, summary report, metrics.
- **Leader** - Peer review: Usually author or none; Formal review (inspection): Trained moderator.
- **Cost / time** - Peer review: Low, quick (30-60 min); Formal review (inspection): Higher, longer.
- **Best used for** - Peer review: Day-to-day code changes; Formal review (inspection): Critical or high-risk work products (payment, security).
- **E-commerce example** - Peer review: Developer A reviews Developer B's product-search code in a pull request; Formal review (inspection): Complete team inspection of payment gateway integration before release.

4.2 Peer-Review Procedure for the "Shopping Cart and Checkout" Module

Team: Dev A (author), Dev B, Dev C, Dev D, Dev E. The Shopping Cart and Checkout module is critical because it affects price calculation, discounts, taxes and orders.

- **1. Submit** - Activity: Author finishes the code, runs unit tests and static analysis, and raises a pull request (max 400 LOC per request) with description and linked requirement; Responsible: Dev A.
- **2. Assign** - Activity: Two peers are assigned (e.g. Dev B for logic, Dev C for security/performance). Review to be completed in 24 hours; Responsible: Team lead / tool.
- **3. Identify defects** - Activity: Reviewers read the code with a checklist (logic, calculation of price/tax/discount, validation, error handling, naming, security, performance). They also run the code locally if necessary; Responsible: Dev B, Dev C.
- **4. Record** - Activity: Every defect is logged as a comment in the review tool (GitHub/Jira) with file, line, description, severity and suggestion; Responsible: Reviewers.
- **5. Discuss** - Activity: A short 20-30 minute meeting or online thread; author clarifies, reviewers agree on valid defects. Disagreements go to the team lead; Responsible: All.
- **6. Correct** - Activity: Author fixes accepted defects in the same branch and replies to each comment with the commit reference; Responsible: Dev A.
- **7. Re-verify** - Activity: Reviewers re-check each fix, rerun unit and regression tests. Only after approval of both reviewers the code is merged; Responsible: Dev B, Dev C.
- **8. Close** - Activity: Defects marked closed; review metrics updated (defects found, time spent); Responsible: Team lead.

4.3 How Defects Are Handled

- **Identify** - Method: Checklist-based reading + running tests; Example: Discount applied before tax instead of after tax.
- **Record** - Method: Tool comment with ID, location, severity, description; Example: DEF-12, Cart.java line 140, Major, wrong total when quantity is 0.
- **Discuss** - Method: Short meeting / thread; focus on whether it is a defect, not on blaming; Example: Confirm that quantity 0 should remove the item.
- **Correct** - Method: Author fixes, adds unit test for the defect; Example: Added test for quantity 0 and negative quantity.
- **Re-verify** - Method: Reviewer checks the fix and regression tests pass; Example: DEF-12 status changed from "Fixed" to "Closed".

4.4 Rules for Healthy Peer Review

- Review the code, not the person.
- Keep changes small so reviews are effective.
- Every critical module change needs at least two approvals.
- Track recurring defects and update the team checklist.

Question 5

A software company has asked its development team to follow strict coding standards and guidelines. Prepare a detailed coding-review checklist containing at least 20 important criteria related to naming conventions, code structure, error handling, security, performance, maintainability, comments, exception handling, input validation, and resource management. Explain why each criterion is important.

Answer

A coding standard is a set of rules about how code should be written in a project: naming, layout, comments, error handling, security and so on. A code review checklist helps the reviewer to check these rules in a systematic way, so that no point is forgotten and every reviewer applies the same criteria. Following standards improves readability, reduces defects, makes maintenance cheaper and helps new

team members to understand the code quickly. The checklist given below has 24 criteria; for each criterion the reason for its importance is also given.

The following checklist contains 24 criteria. Reviewers mark each as Pass / Fail / Not applicable.

- **1** - Category: Naming; Checklist criterion: Variables, methods and classes have meaningful, descriptive names (e.g. totalAmount, not x); Why it is important: Makes code self-explanatory and easy to understand.
- **2** - Category: Naming; Checklist criterion: One naming convention is followed throughout (camelCase, PascalCase, UPPER_CASE for constants); Why it is important: Consistency reduces confusion and review time.
- **3** - Category: Naming; Checklist criterion: No misleading, abbreviated or single-letter names (except loop counters); Why it is important: Avoids misunderstanding and wrong usage.
- **4** - Category: Structure; Checklist criterion: Each function does one task and is short (e.g. under 50 lines); Why it is important: Easy to test, reuse and debug.
- **5** - Category: Structure; Checklist criterion: Code is modular; no duplicated code (DRY principle); Why it is important: A bug fix has to be done in only one place.
- **6** - Category: Structure; Checklist criterion: Nesting of loops/conditions is limited (not more than 3 levels) and cyclomatic complexity is under 10; Why it is important: Complex code is hard to understand and has more defects.
- **7** - Category: Structure; Checklist criterion: Proper indentation, formatting and file organisation are used; Why it is important: Improves readability.
- **8** - Category: Structure; Checklist criterion: No dead code, unused variables, unused imports or commented-out blocks; Why it is important: Reduces clutter and confusion.
- **9** - Category: Structure; Checklist criterion: No magic numbers or hard-coded strings; constants or configuration are used; Why it is important: Easy change and clear meaning.
- **10** - Category: Comments; Checklist criterion: Every class and public method has a header comment (purpose, parameters, return value); Why it is important: Helps new developers and API users.
- **11** - Category: Comments; Checklist criterion: Comments explain "why" for complex logic and are up to date with the code; Why it is important: Outdated comments mislead.
- **12** - Category: Error handling; Checklist criterion: All return codes / error conditions are checked and handled; Why it is important: Prevents silent failures.
- **13** - Category: Error handling; Checklist criterion: Meaningful error messages are produced and logged without exposing sensitive details to users; Why it is important: Helps debugging and avoids information leakage.
- **14** - Category: Exception handling; Checklist criterion: Specific exceptions are caught (not a blank catch-all) and are never swallowed silently; Why it is important: Hidden exceptions make defects hard to find.
- **15** - Category: Exception handling; Checklist criterion: finally / try-with-resources is used for cleanup; exceptions do not leave the system in inconsistent state; Why it is important: Ensures stability and data integrity.
- **16** - Category: Input validation; Checklist criterion: All external input (forms, files, APIs) is validated for type, length, range and format; Why it is important: Prevents invalid data and many attacks.
- **17** - Category: Input validation; Checklist criterion: Validation is done on the server side, not only on the client side; Why it is important: Client-side checks can be bypassed.
- **18** - Category: Security; Checklist criterion: Parameterised queries / prepared statements are used (no SQL string concatenation); Why it is important: Prevents SQL injection.
- **19** - Category: Security; Checklist criterion: No hard-coded passwords/keys; passwords are hashed; sensitive data is encrypted; Why it is important: Protects confidential data.

- **20** - Category: Security; Checklist criterion: Authorisation checks are applied to every protected function; output is encoded; Why it is important: Prevents privilege abuse and XSS.
- **21** - Category: Resource management; Checklist criterion: Files, database connections, sockets and streams are always closed; no memory leaks; Why it is important: Avoids resource exhaustion and crashes.
- **22** - Category: Performance; Checklist criterion: Efficient algorithms and data structures used; no unnecessary loops or repeated database calls inside loops; Why it is important: Keeps response time acceptable under load.
- **23** - Category: Performance; Checklist criterion: Large data sets use pagination/caching; objects are not created needlessly; Why it is important: Saves memory and CPU.
- **24** - Category: Maintainability; Checklist criterion: Code follows design principles (SOLID), is covered by unit tests and has loose coupling; Why it is important: Makes future changes cheap and safe.

Question 6

A Student Result Management System accepts marks between 0 and 100 for five subjects. The system calculates total marks, percentage, grade, and pass/fail status. Design a comprehensive Equivalence Partitioning strategy for testing this system. Identify valid and invalid partitions, boundary-related partitions, representative test data, expected results, and explain why your selected test cases are sufficient.

Answer

Equivalence Partitioning (EP) is a black-box test design technique in which the input domain is divided into groups (partitions) of data that the system should treat in the same way. If one value of a partition works, the other values of that partition are assumed to work also, and if one fails, the others are expected to fail. Therefore only one representative value from each partition needs to be tested. This reduces the number of test cases greatly. Both valid and invalid partitions are tested, and Boundary Value Analysis is combined with EP because defects usually occur at the edges of the partitions.

6.1 Assumptions (needed because the problem does not state the rules)

- Marks are whole numbers from 0 to 100 for each of the 5 subjects.
- Total = sum of 5 marks (maximum 500). Percentage = Total / 5.
- A student passes only if marks in **every** subject are 40 or more.
- Grade by percentage: A+ (90-100), A (80-89), B (70-79), C (60-69), D (50-59), E (40-49), F (below 40 or failed in any subject).

6.2 Equivalence Partitions for a Single Subject Mark

ID	Partition	Range / class	Type	Representative value
V1	Fail marks	0 - 39	Valid	20
V2	Pass, grade E	40 - 49	Valid	45
V3	Grade D	50 - 59	Valid	55
V4	Grade C	60 - 69	Valid	65
V5	Grade B	70 - 79	Valid	75
V6	Grade A	80 - 89	Valid	85
V7	Grade A+	90 - 100	Valid	95
I1	Below minimum	less than 0	Invalid	-5
I2	Above maximum	greater than 100	Invalid	150

ID	Partition	Range / class	Type	Representative value
I3	Non-numeric	letters / special characters	Invalid	abc, @#
I4	Decimal number	e.g. 55.5 (if only integers allowed)	Invalid	55.5
I5	Empty / blank	no value entered	Invalid	(blank)

6.3 Boundary-Related Partitions

Boundaries of valid ranges are where most defects occur (< written instead of <=). The following boundary values are added to the representative values:

- **Lower limit** - Values to test: -1, 0, 1; Reason: Is 0 accepted? Is -1 rejected?.
- **Pass boundary** - Values to test: 39, 40, 41; Reason: Fail/pass decision at 40.
- **Grade boundaries** - Values to test: 49/50, 59/60, 69/70, 79/80, 89/90; Reason: Wrong grade assignment.
- **Upper limit** - Values to test: 99, 100, 101; Reason: Is 100 accepted? Is 101 rejected?.
- **Percentage boundaries** - Values to test: Total 449/450 (89.8/90%), 199/200 (39.8/40%); Reason: Calculation and grade boundary on percentage.

6.4 Test Cases

Format: marks of Subject 1-5.

TC	Input (S1,S2,S3,S4,S5)	Partition covered	Expected result
1	75, 80, 65, 90, 70	V5, V6, V4, V7, V5 (all valid)	Total 380, 76%, Grade B, PASS
2	100, 100, 100, 100, 100	Upper boundary V7	Total 500, 100%, A+, PASS
3	0, 0, 0, 0, 0	Lower boundary V1	Total 0, 0%, F, FAIL
4	40, 40, 40, 40, 40	Pass boundary V2	Total 200, 40%, E, PASS
5	39, 39, 39, 39, 39	Just below pass	Total 195, 39%, F, FAIL
6	50, 50, 50, 50, 50	V3 boundary	Total 250, 50%, D, PASS
7	60, 60, 60, 60, 60	V4 boundary	Total 300, 60%, C, PASS
8	90, 90, 90, 90, 90	V7 boundary	Total 450, 90%, A+, PASS
9	89, 89, 89, 89, 89	Just below A+	Total 445, 89%, A, PASS
10	90, 85, 80, 35, 88	One subject in V1, rest valid	Total 378, 75.6%, FAIL (subject below 40)
11	-1, 60, 60, 60, 60	I1 (below minimum)	Error: marks must be between 0 and 100; no result
12	60, 101, 60, 60, 60	I2 (above maximum)	Error message; no result
13	60, abc, 60, 60, 60	I3 (non-numeric)	Error: enter numeric value
14	60, 60, 55.5, 60, 60	I4 (decimal)	Error or as per specification
15	60, 60, (blank), 60, 60	I5 (blank)	Error: marks required
16	1, 100, 99, 0, 50	Mixed boundaries	Total 250, 50%, FAIL (subjects 1 and 4 below 40)

6.5 Why These Test Cases Are Sufficient

- Every valid and invalid partition is covered by at least one test case (100% partition coverage).
- Each invalid partition is tested one at a time so that the cause of any failure is clear.
- Boundary values catch off-by-one errors in the range check, pass/fail rule and grade calculation.
- Cases 1-9 cover all grades, and cases 5 and 10 verify that pass/fail depends on each subject and not only on the percentage.

- All members of one partition are expected to behave identically, so testing one representative gives the same defect-finding ability as testing all 101 possible marks. Combined with BVA this gives a small, effective suite instead of 101^5 combinations.

Question 7

An online shopping application provides the following discount rules: below Rs. 1,000: no discount; Rs. 1,000 - 4,999: 5% discount; Rs. 5,000 - 9,999: 10% discount; Rs. 10,000 - 49,999: 15% discount; Rs. 50,000 or above: 20% discount. Design an extensive set of test cases using Equivalence Partitioning and Boundary Value Analysis. Identify potential defects that could occur at boundary values and explain how your test cases would expose them.

Answer

Equivalence Partitioning divides the purchase amount into ranges, and every range gets the same discount, so one value from each range is enough to represent it. Boundary Value Analysis (BVA) is based on the experience that defects are most common at the edges of ranges, for example because a programmer writes > instead of >=. In BVA the minimum, maximum and values just below and just above each boundary are tested. In this problem the boundaries are Rs. 1,000, 5,000, 10,000 and 50,000.

7.1 Equivalence Partitions (purchase amount in Rs.)

ID	Partition	Type	Discount	Representative value
P1	Less than 0 (negative)	Invalid	Error	-500
P2	0 (zero)	Invalid / special	Error or no purchase	0
P3	0.01 - 999.99	Valid	0%	500
P4	1,000 - 4,999.99	Valid	5%	3,000
P5	5,000 - 9,999.99	Valid	10%	7,500
P6	10,000 - 49,999.99	Valid	15%	25,000
P7	50,000 and above	Valid	20%	75,000
P8	Non-numeric / blank	Invalid	Error	abc, blank

7.2 Boundary Value Test Cases

Final amount = Amount - Discount. Boundaries: 1,000; 5,000; 10,000; 50,000 (each tested just below, on, and just above).

TC	Purchase amount (Rs.)	Partition / boundary	Expected discount	Expected payable (Rs.)
1	-500	P1 invalid	Error message	Not processed
2	0	P2	Error / no order	Not processed
3	1	Minimum valid	0%	1.00
4	500	P3 representative	0%	500.00
5	999	Upper boundary of P3	0%	999.00
6	999.99	Just below 1,000	0%	999.99
7	1,000	Lower boundary of P4	5%	950.00
8	1,001	Just above 1,000	5%	950.95
9	3,000	P4 representative	5%	2,850.00
10	4,999	Upper boundary of P4	5%	4,749.05

TC	Purchase amount (Rs.)	Partition / boundary	Expected discount	Expected payable (Rs.)
11	5,000	Lower boundary of P5	10%	4,500.00
12	5,001	Just above 5,000	10%	4,500.90
13	7,500	P5 representative	10%	6,750.00
14	9,999	Upper boundary of P5	10%	8,999.10
15	10,000	Lower boundary of P6	15%	8,500.00
16	10,001	Just above 10,000	15%	8,500.85
17	25,000	P6 representative	15%	21,250.00
18	49,999	Upper boundary of P6	15%	42,499.15
19	50,000	Lower boundary of P7	20%	40,000.00
20	50,001	Just above 50,000	20%	40,000.80
21	75,000	P7 representative	20%	60,000.00
22	9,99,99,999 (very large)	Extreme value	20%	80% of amount, no overflow
23	abc / blank	P8 invalid	Error message	Not processed
24	4,999.50	Gap between 4,999 and 5,000	5% (as per clarified requirement)	4,749.53 (rounded)

7.3 Potential Boundary Defects and How the Test Cases Expose Them

- **Wrong relational operator (> instead of >=)** - Example in code: if (amt > 1000) gives 0% for exactly 1,000; Test cases that expose it: TC 7, 11, 15, 19 (exact boundary values).
- **Wrong operator on upper side (<= instead of <)** - Example in code: if (amt <= 5000) gives 5% for exactly 5,000; Test cases that expose it: TC 11 expects 10% but defective code gives 5%.
- **Off-by-one error** - Example in code: Comparing with 999 or 1001 instead of 1000; Test cases that expose it: TC 5, 6, 7, 8.
- **Gap or overlap in ranges** - Example in code: Amount 4,999.50 matches no rule or two rules; Test cases that expose it: TC 24, 6, 10.
- **Decimal / rounding error** - Example in code: Floating point calculation gives 949.9999; Test cases that expose it: TC 8, 10, 12, 14, 16, 18, 20 (check 2-decimal accuracy).
- **Discount applied cumulatively instead of one slab** - Example in code: Customer with 12,000 gets 5%+10%+15%; Test cases that expose it: TC 15-18 verify only a single discount.
- **Wrong percentage assigned to slab** - Example in code: Table values swapped; Test cases that expose it: Representative values TC 9, 13, 17, 21.
- **Data type overflow / limit** - Example in code: int used for large amounts; Test cases that expose it: TC 22.
- **Invalid input accepted** - Example in code: Negative amount gives a discount; Test cases that expose it: TC 1, 2, 23.
- **Discount calculated on amount after tax or shipping by mistake** - Example in code: Rule says purchase value; Test cases that expose it: Verify against the specification with the same boundary cases.

Conclusion: the suite has 24 test cases: 3 for invalid input, 4 representative cases (one per valid slab) and the remaining cases on both sides of each boundary, so that every partition and every boundary is covered.

Question 8

Explain Test to Pass and Test to Fail using a realistic ATM withdrawal system. Design separate test scenarios for both approaches. Your answer must include normal conditions, invalid inputs, boundary conditions, insufficient balance, invalid PIN, blocked card, network failure, and transaction timeout situations.

Answer

8.1 Concepts

- **Checks that the software does what it is supposed to do under normal and valid conditions.:** Tries to make the software fail by using invalid inputs, extreme conditions and error situations.
- **Uses valid data and expected user behaviour.:** Uses invalid, unexpected, boundary and stress conditions.
- **Done first to confirm basic functionality.:** Done after basic tests pass, to find robustness and security defects.
- **Example: valid card, correct PIN and sufficient balance dispenses cash.:** Example: wrong PIN three times, card gets blocked and cash is not dispensed.

8.2 Assumptions for the ATM

- Withdrawal amount must be a multiple of Rs. 100, minimum Rs. 100, maximum Rs. 10,000 per transaction and Rs. 40,000 per day.
- PIN has 4 digits; card is blocked after 3 consecutive wrong PINs.
- Minimum account balance Rs. 500 must be kept.
- ATM session times out after 30 seconds of inactivity.

8.3 Test to Pass Scenarios

TC	Scenario	Test data / steps	Expected result
P1	Normal withdrawal	Valid card, correct PIN, balance Rs. 20,000, withdraw Rs. 5,000	Cash dispensed, balance Rs. 15,000, receipt printed.
P2	Minimum amount (boundary)	Withdraw Rs. 100	Cash dispensed.
P3	Maximum per transaction (boundary)	Withdraw Rs. 10,000, balance sufficient	Cash dispensed.
P4	Withdraw all but minimum balance	Balance 5,000, withdraw 4,500	Cash dispensed, balance 500.
P5	Valid PIN on 3rd attempt	Wrong PIN twice, correct third time	Access granted; wrong counter reset.
P6	Multiple withdrawals within daily limit	Four withdrawals of Rs. 10,000	All successful (Rs. 40,000).
P7	Card returned after transaction	Complete transaction and choose "No more transaction"	Card ejected, session ended.
P8	Mini statement after withdrawal	Select print receipt/mini statement	Correct amount and balance printed.

8.4 Test to Fail Scenarios

TC	Condition	Test data / steps	Expected result (system must fail safely)
F1	Invalid input: not multiple of 100	Enter Rs. 550	Message "Enter amount in multiples of 100"; no cash.
F2	Invalid input: zero / negative / non-numeric	Enter 0, -500, "abc", blank	Input rejected with message.
F3	Boundary: below minimum	Enter Rs. 99 or Rs. 50	Rejected.

TC	Condition	Test data / steps	Expected result (system must fail safely)
F4	Boundary: above maximum per transaction	Enter Rs. 10,100 and Rs. 10,001	Rejected: exceeds transaction limit.
F5	Daily limit exceeded	Fifth withdrawal after Rs. 40,000 already withdrawn	Rejected: daily limit reached.
F6	Insufficient balance	Balance 2,000, withdraw 3,000	Rejected: insufficient balance; balance unchanged.
F7	Minimum balance violation	Balance 1,000, withdraw 600	Rejected as balance would fall below 500.
F8	Invalid PIN once	Enter wrong PIN	Error message and retry allowed; attempts counter = 1.
F9	Invalid PIN three times	Enter wrong PIN 3 times	Card blocked/retained; message to contact the bank.
F10	Blocked card	Insert already blocked card	Card rejected/retained; no PIN prompt for transaction.
F11	Expired or damaged card	Use expired card, scratched chip/strip	Card rejected with message; card returned.
F12	Network failure before authorisation	Disconnect network when amount is submitted	Transaction cancelled; no cash; no debit; message displayed.
F13	Network failure after debit, before dispensing	Network lost after bank approved the debit	Automatic reversal within defined time; transaction logged; customer gets failure slip.
F14	Transaction timeout	Do not press any key for 30+ seconds at amount screen	Session ends, card returned, no debit.
F15	Timeout after cash presented, not taken	Do not take cash for 20 seconds	Cash retracted and amount reversed.
F16	ATM out of cash / wrong denominations	Request amount that ATM cannot dispense	Appropriate message; no debit.
F17	Power failure during dispensing	Simulate power cut	Consistency check after restart; reversal if cash not dispensed.
F18	Cancel button during transaction	Press cancel after entering amount	Transaction cancelled, card returned.

Conclusion: Test to Pass proves that the ATM works for valid users; Test to Fail proves that it stays safe, correct and secure when things go wrong. Both are needed since in banking, error handling is as important as normal flow.

Question 9

A hospital appointment system allows patients to enter their age, select a doctor, choose an appointment date, and make an online payment. Design a complete Data Testing strategy for this application. Include valid data, invalid data, boundary data, missing data, incorrect data types, duplicate data, inconsistent data, extreme values, and malicious/unexpected input.

Answer

Data testing checks whether the application accepts, processes, stores and displays data correctly. It covers the validation of the input fields, the handling of wrong or hostile data, and the integrity of data in the database. For a hospital appointment system, data errors are serious: a wrong age may select the wrong doctor, a duplicate booking blocks a slot of another patient, and a failed payment with a confirmed appointment causes a financial dispute. The strategy below therefore tests each type of data

on all four fields (age, doctor, date and payment) at the user interface level, the server level and the database level.

9.1 Assumptions

- Valid age is 0 to 120 years (children below 18 require guardian details).
- Appointment date can be from today up to 60 days in the future.
- A doctor has limited slots per day and works only on certain days.
- Consultation fee is fixed per doctor; payment by card/UPI/net banking.

9.2 Data Test Strategy by Category

- **Valid data** - Test data / scenario: Age 35; doctor "Cardiologist - Dr. Sharma"; date tomorrow (doctor available); successful UPI payment; Expected result: Appointment booked, confirmation message and e-mail/SMS sent.
- **Valid data (typical others)** - Test data / scenario: Age 8 with guardian, age 70 senior citizen; weekday slots; different payment modes; Expected result: Booked correctly; any concession applied properly.
- **Boundary data: age** - Test data / scenario: -1, 0, 1, 17, 18, 119, 120, 121; Expected result: -1 and 121 rejected; 0 accepted (infant) or as per rule; 17 asks guardian detail.
- **Boundary data: date** - Test data / scenario: Yesterday, today (slot time already past/not past), tomorrow, day 60, day 61; Expected result: Past date rejected; today valid only for future slots; day 61 rejected.
- **Boundary data: slots** - Test data / scenario: Last available slot, slot when doctor is full; Expected result: Last slot booked successfully; next request shows "No slots available".
- **Missing data** - Test data / scenario: Age blank, doctor not selected, date blank, payment details empty; Expected result: Mandatory field messages; form not submitted; no payment initiated.
- **Incorrect data type** - Test data / scenario: Age = "twenty", "3a", 25.5; date = "32/13/2026", "abcd"; amount as text; Expected result: Validation error; no server error / crash.
- **Invalid data** - Test data / scenario: Age -5 or 500; date in the past; doctor ID that does not exist; unavailable doctor on Sunday; Expected result: Rejected with clear message.
- **Duplicate data** - Test data / scenario: Same patient books same doctor, same date and time twice; double-click on Pay; browser refresh after payment; two patients booking same last slot simultaneously; Expected result: Duplicate prevented; only one booking and one payment; second user gets slot unavailable message.
- **Inconsistent data** - Test data / scenario: Age 5 selecting a geriatrician; age 70 selecting a paediatrician; appointment date earlier than registration date; payment amount not equal to doctor fee; payment success but appointment not saved; Expected result: System warns or blocks as per rule; amount verified on server; transaction and booking remain consistent (rollback / auto-refund).
- **Extreme values** - Test data / scenario: Age 99999999999; very long name (5,000 chars); date year 9999 or 1900; 10,000 bookings together; amount 0 or 99,99,999; Expected result: Rejected gracefully; no overflow, truncation or timeout.
- **Malicious input** - Test data / scenario: SQL injection: age = "1 OR 1=1", name = "'; DROP TABLE patients;--"; XSS: <script>alert(1)</script> in notes; HTML tags; URL tampering of doctor ID/amount; negative amount through proxy tool; unicode / emoji; special characters; Expected result: Input sanitised or rejected; no script executed; no data change; server-side validation prevents tampering; event logged.
- **Unexpected input** - Test data / scenario: Leading/trailing spaces, copy-paste with hidden characters, different date formats (DD/MM vs MM/DD), different language characters; Expected result: Handled consistently or rejected with clear message.

- **Data integrity / persistence** - Test data / scenario: Booking is stored in database with correct patient, doctor, date and amount; check after logout/login and after system restart; Expected result: Data identical everywhere (UI, DB, e-mail, invoice).
- **Payment data** - Test data / scenario: Wrong card number, expired card, wrong CVV, insufficient funds, gateway timeout, back button after payment; Expected result: Payment failure message; appointment not confirmed; no double deduction.

9.3 Test Approach

- Use Equivalence Partitioning and BVA for age, date and amount fields.
- Verify both client-side and server-side validation (disable JavaScript or use API tools).
- Check data in the database and in generated outputs (receipts, e-mails) after each test.
- Use automated data-driven tests for large sets of valid/invalid combinations.
- Run security scans for injection and XSS.

Question 10

Consider a bank account application having the following states: New Account → Active → Suspended → Active → Closed. An account can become suspended after suspicious activity and can be permanently closed after specific conditions are satisfied. Develop a State Transition Testing model for the application. Identify valid and invalid transitions and prepare detailed test cases for each important transition.

Answer

State Transition Testing is a black-box technique used for systems that behave differently depending on their current state and the events that occur. The system is modelled as a set of states, events (inputs), transitions and actions. Test cases are designed to cover the valid transitions and also to try invalid transitions, which the system must reject without changing the state. A bank account is a good example, because its allowed operations depend completely on its state: a suspended account must not allow withdrawal and a closed account must not allow anything.

10.1 State Model

States: S1 = New Account, S2 = Active, S3 = Suspended, S4 = Closed (final state).

Events: E1 = KYC verified and initial deposit made (activate), E2 = suspicious activity detected, E3 = investigation cleared / reactivation approved, E4 = closure request with all conditions satisfied (zero balance, no pending loans/cheques/dues), E5 = transaction request (deposit, withdraw, transfer).

State flow: New Account --E1--> Active --E2--> Suspended --E3--> Active --E4--> Closed. (Assumption: as per the given model, closure is possible only from Active; if the bank also allows closure from Suspended, this should be clarified in the requirement and the transition added.)

10.2 State Transition Table

Current state	E1 Activate	E2 Suspicious activity	E3 Clear / Reactivate	E4 Close (conditions met)	E5 Transaction
New Account	Active (valid)	Invalid	Invalid	Invalid	Invalid (rejected)
Active	Invalid (already active)	Suspended (valid)	Invalid (nothing to clear)	Closed (valid)	Allowed (stays Active)
Suspended	Invalid	Invalid (already suspended)	Active (valid)	Invalid (per model)	Rejected (stays Suspended)
Closed	Invalid	Invalid	Invalid	Invalid	Invalid (rejected)

10.3 Valid and Invalid Transitions

- **Valid** - Transition: New Account to Active; Remark: On successful KYC and deposit.
- **Valid** - Transition: Active to Suspended; Remark: On suspicious activity.
- **Valid** - Transition: Suspended to Active; Remark: After clearance by compliance officer.
- **Valid** - Transition: Active to Closed; Remark: When closure conditions are satisfied.
- **Invalid** - Transition: New Account to Suspended; Remark: No activity to be suspicious about.
- **Invalid** - Transition: New Account to Closed; Remark: Account never opened; should be cancelled differently.
- **Invalid** - Transition: Suspended to Closed (per given model); Remark: Must first be reactivated, or requirement must be clarified.
- **Invalid** - Transition: Closed to Active / Suspended / New; Remark: Closure is permanent.
- **Invalid** - Transition: Active to New Account / Suspended to New Account; Remark: Backward transitions not permitted.

10.4 Test Cases

TC	Initial state	Event / action	Expected final state and result
1	New Account	Complete KYC and deposit minimum amount	Active; account number enabled; confirmation sent. (Valid)
2	New Account	Activate with incomplete KYC / deposit below minimum	Remains New Account; error shown. (Negative)
3	New Account	Try withdraw/transfer	Rejected; state unchanged. (Invalid)
4	New Account	Mark as suspicious	Rejected; remains New. (Invalid)
5	New Account	Try to close	Rejected or handled as cancellation; not Closed. (Invalid)
6	Active	Normal deposit / withdrawal / transfer	Allowed; state remains Active. (Valid)
7	Active	System detects suspicious activity (e.g. multiple high-value transfers in a minute)	Suspended; customer notified; alert to compliance. (Valid)
8	Active	Activate again	Ignored; remains Active, no duplicate record. (Invalid)
9	Suspended	Customer tries withdrawal / transfer / bill payment	Rejected with message; state remains Suspended. (Negative)
10	Suspended	Incoming deposit / credit	As per business rule (accept or hold); state remains Suspended.
11	Suspended	Compliance approves reactivation	Active; transactions allowed again; audit log updated. (Valid)
12	Suspended	Reactivation by unauthorised user	Rejected; remains Suspended. (Security)
13	Suspended	Second suspicious activity event	Remains Suspended; event logged only. (Invalid)
14	Suspended	Closure request	Rejected as per model; remains Suspended. (Invalid)
15	Active	Closure request with balance > 0 or pending loan	Rejected; remains Active; reason displayed. (Negative)
16	Active	Closure request with all conditions met	Closed; confirmation and final statement sent. (Valid)
17	Closed	Any transaction (deposit, withdraw, transfer)	Rejected; account closed message. (Invalid)
18	Closed	Reactivate / suspend / activate	Rejected; state stays Closed. (Invalid)

TC	Initial state	Event / action	Expected final state and result
19	Active to Suspended to Active to Closed	Complete life cycle in sequence (state-path test)	All transitions work in order; history/audit log complete.
20	Active	Suspicious event occurs during an in-progress transaction	Transaction either completes or rolls back consistently; account becomes Suspended after.

Coverage achieved: all states, all valid transitions (0-switch coverage), and all invalid transitions in the table (invalid transition coverage).

Question 11

A Food Delivery Application has the following order states: Order Placed → Payment Confirmed → Restaurant Accepted → Food Preparing → Out for Delivery → Delivered. The order may also enter Cancelled, Payment Failed, or Refund Initiated. Design a complete state-based testing strategy for this application. Include valid transitions, invalid transitions, unexpected events, repeated events, and recovery scenarios.

Answer

An order in a food delivery application moves through several states, and the next step depends on the current state and on events from three parties: the customer, the restaurant and the delivery partner. State-based testing therefore checks that only the allowed sequence is possible, that wrong sequences are rejected, and that the order remains consistent even when events are repeated, delayed or lost. Because money is involved, the cancellation and refund paths are tested as thoroughly as the normal path.

11.1 States and Events

States: Order Placed (OP), Payment Confirmed (PC), Payment Failed (PF), Restaurant Accepted (RA), Food Preparing (FP), Out for Delivery (OD), Delivered (DL), Cancelled (CN), Refund Initiated (RI).

Events: pay success, pay failure, retry payment, restaurant accept, restaurant reject / timeout, start preparation, pickup by delivery partner, delivery completed, customer cancel, refund started.

11.2 Valid Transitions

- **Order Placed** - Event: Payment successful; To: Payment Confirmed.
- **Order Placed** - Event: Payment fails; To: Payment Failed.
- **Payment Failed** - Event: Retry payment succeeds; To: Payment Confirmed.
- **Payment Failed** - Event: Customer abandons / timeout; To: Cancelled.
- **Payment Confirmed** - Event: Restaurant accepts; To: Restaurant Accepted.
- **Payment Confirmed** - Event: Restaurant rejects or no response in time; To: Cancelled, then Refund Initiated.
- **Restaurant Accepted** - Event: Kitchen starts; To: Food Preparing.
- **Food Preparing** - Event: Delivery partner picks up; To: Out for Delivery.
- **Out for Delivery** - Event: Customer receives order; To: Delivered.
- **Payment Confirmed / Restaurant Accepted** - Event: Customer cancels (allowed window); To: Cancelled, then Refund Initiated.
- **Food Preparing** - Event: Cancel (by restaurant issue, or customer with partial refund); To: Cancelled, then Refund Initiated (full or partial).

- **Out for Delivery** - Event: Delivery failed / customer unreachable / item missing; To: Cancelled or Refund Initiated as per policy.
- **Delivered** - Event: Complaint approved (wrong/spoiled food); To: Refund Initiated.

11.3 Test Strategy and Test Cases

A. Valid transition tests

TC	Scenario	Expected result
V1	Place order, pay successfully	Status Payment Confirmed; restaurant is notified.
V2	Complete happy path: Placed, Paid, Accepted, Preparing, Out for Delivery, Delivered	Each status appears in sequence with timestamp; customer gets notification at every step; Delivered is final.
V3	Cancel after payment but before restaurant accepts	Cancelled, then Refund Initiated for 100%.
V4	Restaurant rejects order	Cancelled; automatic full refund; customer informed.

B. Invalid transition tests

TC	Scenario	Expected result
I1	Order Placed directly to Out for Delivery (skip payment)	Rejected; status unchanged.
I2	Payment Failed to Restaurant Accepted without payment	Rejected; restaurant cannot accept unpaid order.
I3	Delivered to Preparing / Out for Delivery (backward)	Rejected.
I4	Cancelled to Restaurant Accepted / Preparing	Rejected; cancelled is final (except refund).
I5	Customer cancels after Delivered	Not allowed; complaint/refund flow offered instead.
I6	Refund Initiated to Delivered	Rejected.
I7	Food Preparing to Payment Confirmed	Rejected.

C. Unexpected events

TC	Scenario	Expected result
U1	Restaurant does not respond for 5 minutes	Order auto-cancelled and refunded, or reassigned as per policy.
U2	Payment gateway times out but money is deducted	Status remains pending, then reconciled to Confirmed or auto-refund; no lost money.
U3	Delivery partner app crashes or phone off	Order reassigned to another partner; customer informed.
U4	Customer app closed/offline during order	Order continues; status is correct on reopening.
U5	Restaurant marks item out of stock after acceptance	Customer asked to modify or cancel; partial/full refund.
U6	Server failure between two state changes	State remains consistent (atomic update); no half-processed order.
U7	Delivery location changed after Out for Delivery	Rejected or handled as per rules.

D. Repeated events

TC	Scenario	Expected result
R1	Customer double-taps "Pay"	Only one payment is made (idempotent).

TC	Scenario	Expected result
R2	Payment success callback received twice from gateway	Second ignored; no duplicate state change or duplicate notification.
R3	Restaurant presses "Accept" twice	Only one acceptance; status unchanged.
R4	Customer presses Cancel twice	Only one cancellation; only one refund.
R5	Delivery partner marks "Delivered" twice	Only one delivery record.
R6	Refund request submitted repeatedly	Only one refund is issued.

E. Recovery scenarios

TC	Scenario	Expected result
RC1	Payment failed, customer retries with same order	Order moves to Payment Confirmed without creating a duplicate order.
RC2	Payment failed, retry also fails, then user changes payment mode	Eventually succeeds or order is cancelled; no double charge.
RC3	Refund fails at bank	Retry mechanism; status shows Refund Initiated/Pending; customer support alerted.
RC4	App/server restart in the middle of order	Order state restored from database; no data loss.
RC5	Delivery partner cancelled after pickup	New partner assigned; customer notified; delivered later with delay notice.
RC6	Order stuck in one state beyond time limit (e.g. Food Preparing > 60 min)	Escalation alert/auto-cancel; full refund and apology voucher.

Coverage: all states, all valid transitions, all invalid pairs, repeated and abnormal events - achieving state, transition and negative-transition coverage.

Question 12

Explain how Decision Table Testing can be applied to an insurance claim processing system. Assume that claim approval depends on policy validity, premium payment status, accident coverage, document availability, and claim amount. Construct a suitable decision table and derive test cases from it.

Answer

Decision Table Testing is a black-box technique for systems in which the result depends on combinations of conditions, which is common in business rules. A decision table has conditions in the upper part, actions in the lower part and one column (rule) for each combination of conditions. Every rule is turned into at least one test case. In an insurance claim system the decision depends on several factors together, and testing them one by one would miss combinations, so decision tables are a suitable technique.

12.1 Decision Table Testing

Decision table testing is a black-box technique used when the output depends on a combination of conditions. The conditions and actions are listed in a table, each column (rule) represents one unique combination, and each rule gives at least one test case. It ensures that no combination of business rules is missed, and it exposes missing or contradicting rules in the requirement.

12.2 Conditions and Actions

- **C1** - Condition: Policy is valid (active, not expired); Values: Y / N.

- **C2** - Condition: Premium paid up to date; Values: Y / N.
- **C3** - Condition: Accident / event is covered under the policy; Values: Y / N.
- **C4** - Condition: All required documents available; Values: Y / N.
- **C5** - Condition: Claim amount is within the coverage limit; Values: Y / N.

Actions: A1 = Approve full claim, A2 = Approve partial claim (up to limit), A3 = Reject claim, A4 = Hold claim and request documents.

With 5 binary conditions there are 2^5 = 32 possible combinations. Many are not independent: if the policy is invalid or the premium is not paid, the other conditions do not matter. The table is therefore reduced using "don't care" (-) entries, giving 6 rules.

12.3 Decision Table

Conditions / Actions	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**
C1 Policy valid	N	Y	Y	Y	Y	Y
C2 Premium paid	-	N	Y	Y	Y	Y
C3 Accident covered	-	-	N	Y	Y	Y
C4 Documents available	-	-	-	N	Y	Y
C5 Amount within limit	-	-	-	-	N	Y
A1 Approve full						X
A2 Approve partial					X	
A3 Reject	X	X	X			
A4 Hold / request documents				X		

12.4 Test Cases Derived from the Table

TC	**Rule**	**Input**	**Expected result**
1	R1	Policy expired, premium paid, covered accident, documents complete, amount Rs. 50,000	Claim rejected - policy invalid.
2	R2	Policy valid, premium unpaid (lapsed), covered accident, documents complete, amount Rs. 50,000	Claim rejected - premium not paid.
3	R3	Policy valid, premium paid, accident type not covered (e.g. intentional damage), documents complete	Claim rejected - not covered.
4	R4	Policy valid, premium paid, covered accident, FIR/medical bill missing	Claim on hold; request for documents sent.
5	R5	All valid, covered, documents complete, claim Rs. 6,00,000 while limit is Rs. 5,00,000	Partial approval of Rs. 5,00,000 (or manual review as per rule).
6	R6	All valid, covered, documents complete, claim Rs. 3,00,000 within limit	Full claim approved.
7	Extra (BVA)	Same as R6, claim exactly equal to limit Rs. 5,00,000	Full approval (boundary inclusive).
8	Extra (BVA)	Same as R6, claim Rs. 5,00,001	Partial approval of Rs. 5,00,000.
9	Extra (boundary)	Policy expiry date equal to accident date	Valid per rule (verify with requirement).
10	Extra (multiple failures)	Policy invalid AND documents missing	Rejected (rule R1 takes priority over request for documents).

12.5 Benefits

- Systematic coverage of all rule combinations with minimum test cases.

- Highlights ambiguous or missing rules (e.g. what if documents are missing and amount exceeds limit?).
- Easy for business users and testers to review together.

Question 13

A login system has the following rules: Username must be registered. Password must be correct. Account must be active. Account must not be locked. CAPTCHA must be successfully verified. Analyze this system using Decision Table Testing. Construct the decision table, identify all meaningful combinations of conditions, and derive a minimum but effective set of test cases.

Answer

A login system is a typical example for a decision table because the final result (success or a particular error message) depends on the combination of five checks. With 5 binary conditions there are 32 combinations, but not all of them need a separate test, because once one check fails the others usually do not change the result. The decision table below finds the meaningful combinations, and the test cases are derived from them.

13.1 Conditions and Actions

- **C1** - Condition: Username is registered; Values: Y / N.
- **C2** - Condition: Password is correct; Values: Y / N.
- **C3** - Condition: Account is active; Values: Y / N.
- **C4** - Condition: Account is not locked; Values: Y / N.
- **C5** - Condition: CAPTCHA verified successfully; Values: Y / N.

Actions: A1 = Login successful, A2 = "Invalid username or password" message, A3 = "Account inactive" message, A4 = "Account locked" message, A5 = "CAPTCHA failed" message.

Five conditions give 32 combinations. Only one combination (all Y) gives successful login. In the other 31, the system must deny access. They are grouped by the first failing check (assuming the order: CAPTCHA, username, password, active, locked) which gives the meaningful combinations below.

13.2 Decision Table (reduced)

Conditions / Actions	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**
C1 Username registered	Y	N	Y	Y	Y	Y
C2 Password correct	Y	-	N	Y	Y	Y
C3 Account active	Y	-	-	N	Y	Y
C4 Account not locked	Y	-	-	-	N	Y
C5 CAPTCHA verified	Y	Y	Y	Y	Y	N
A1 Login success	X					
A2 Invalid username/password		X	X			
A3 Account inactive				X		
A4 Account locked					X	
A5 CAPTCHA failed						X

Note: in R6 the CAPTCHA failure takes precedence over all other conditions, so C1-C4 are tested with valid values (their values do not affect the output).

13.3 Minimum Effective Test Set

TC	Rule	Username	Password	Status	CAPTCHA	Expected result
1	R1	Registered	Correct	Active, not locked	Correct	Login successful; session created.
2	R2	Unregistered	Any	-	Correct	Invalid username or password (no hint which one is wrong).
3	R3	Registered	Wrong	Active	Correct	Invalid username or password; failed-attempt counter increases.
4	R4	Registered	Correct	Inactive / disabled	Correct	Account inactive message.
5	R5	Registered	Correct	Locked	Correct	Account locked message; contact support / unlock after time.
6	R6	Registered	Correct	Active, not locked	Wrong / expired	CAPTCHA failed; new CAPTCHA displayed; login denied.
7	Priority check	Unregistered	Wrong	-	Wrong	CAPTCHA failed (as per stated order); no information leakage.
8	Priority check	Registered	Wrong	Locked	Correct	Account locked (locked check before password) - confirm with requirement.
9	Lock rule	Registered	Wrong 3 times (as per policy)	Active	Correct	Account becomes locked after limit; next correct login also denied.

Justification: TC1-TC6 cover each rule of the table (each failing condition once, and the success path). TC7-TC9 are added to remove ambiguity about precedence when more than one condition fails at the same time; therefore 9 test cases are sufficient and minimal for effective coverage instead of 32.

Question 14

A software application is required to accept passwords satisfying the following conditions: Minimum 8 characters; Maximum 20 characters; At least one uppercase letter; At least one lowercase letter; At least one digit; At least one special character. Develop a black-box test suite using Equivalence Partitioning, Boundary Value Analysis, and negative testing. Explain which defects each category of test cases is intended to detect.

Answer

For the password field the specification has two kinds of rules: a length rule (8 to 20) and four character-type rules. Equivalence Partitioning is applied to each rule, Boundary Value Analysis is applied to the length, and negative testing tries unexpected input to see whether the system fails safely. In the test cases, only one rule is violated at a time, so that a failure can be traced to the exact rule that is wrong.

14.1 Equivalence Partitioning

ID	Partition	Type
E1	Length 8 to 20	Valid
E2	Length less than 8	Invalid
E3	Length more than 20	Invalid
E4	Contains uppercase	Valid

ID	Partition	Type
E5	No uppercase letter	Invalid
E6	Contains lowercase	Valid
E7	No lowercase letter	Invalid
E8	Contains digit	Valid
E9	No digit	Invalid
E10	Contains special character	Valid
E11	No special character	Invalid

TC	Password	Length	Partitions	Expected result
EP1	Welcome@2026x	13	E1, E4, E6, E8, E10	Accepted
EP2	Ab1@xyz	7	E2	Rejected - too short
EP3	Abcdefghijklmnopqrstuv1@	24	E3	Rejected - too long
EP4	abcdefg1@	9	E5	Rejected - no uppercase
EP5	ABCDEFG1@	9	E7	Rejected - no lowercase
EP6	Abcdefgh@	9	E9	Rejected - no digit
EP7	Abcdefg12	9	E11	Rejected - no special character

14.2 Boundary Value Analysis (length)

TC	Password	Length	Expected result
BV1	Abcde1@	7	Rejected (just below minimum)
BV2	Abcdef1@	8	Accepted (minimum)
BV3	Abcdefg1@	9	Accepted (just above minimum)
BV4	Abcdefghijklmnopq1@	19	Accepted (just below maximum)
BV5	Abcdefghijklmnopqr1@	20	Accepted (maximum)
BV6	Abcdefghijklmnopqrs1@	21	Rejected (just above maximum)

14.3 Negative Testing

TC	Input	Expected result
N1	Empty password	Rejected - required field
N2	Null / field not sent in request	Rejected - no server error
N3	Only spaces (8 spaces)	Rejected
N4	Only letters / only digits / only special characters (e.g. 12345678, ABCDEFGH, !@#$%^&*)	Rejected - missing required types
N5	Two types missing (e.g. abcdefgh, ABCDEFG1)	Rejected; message lists missing rules
N6	Very long input of 1,000+ characters	Rejected gracefully; no crash or buffer problem
N7	Leading/trailing space: " Abcdef1@ "	As per specification: trimmed or rejected; consistent behaviour
N8	Unicode / non-English characters: "Pässwörd1@", emoji	Behaviour as specified; no corruption in storage
N9	Injection strings: "Abc1@' OR 1=1--" and "<script>alert(1)</script>Aa1@"	Treated as plain text (hashed); no script/SQL executed

TC	Input	Expected result
N10	Common/weak or username-containing password: "Password1@"	Accepted/rejected according to policy (recommend rejection)
N11	Password change: new password same as old	As per policy
N12	Paste with hidden characters / newline in password	Rejected or cleaned consistently

14.4 Defects Each Category Is Intended to Detect

- **Equivalence Partitioning:** Missing validation for a rule (e.g. special character check not implemented); wrong error message; validation of one rule overriding another; rules combined with wrong logic (OR instead of AND).
- **Boundary Value Analysis:** Off-by-one defects in length check (< 8 vs <= 8, > 20 vs >= 20); wrong limit constants; database column too short (e.g. VARCHAR(15)) truncating 20 character passwords.
- **Negative testing:** Crashes on null/empty/long input; weak handling of spaces and unusual characters; security flaws such as SQL injection or XSS; inconsistent trimming; information leakage in error messages; improper storage (unhashed) of special characters.

Question 15

Explain the role of static black-box testing in software development. Select a realistic requirement document for an online shopping system and identify ambiguous, incomplete, contradictory, unverifiable, and missing requirements. Prepare a review checklist that could be used by a testing team before implementation begins.

Answer

Static black-box testing is the examination of the product specification (requirements document, user stories, prototypes) without executing any code. It is called black-box because the tester looks only at the externally visible behaviour that is specified, not at the internal design. Studies show that a large share of software defects come from the specification, so checking it before coding is very effective. The tester reads the document like a customer and also like a tester, and asks whether each statement is clear, complete, consistent, testable and necessary.

15.1 Role of Static Black-Box Testing

Static black-box testing means examining the specification or requirement document (without running any program and without looking at the code) to find defects in the specification itself. The tester treats the system as a black box and checks whether the documented behaviour is clear, complete and testable.

- **Finds defects at the cheapest stage:** most major software defects originate in poor specifications.
- **Improves testability:** requirements become precise enough to derive test cases and expected results.
- **Reduces rework:** developers do not build wrong or unclear features.
- **Establishes shared understanding** between customer, analyst, developer and tester.
- **Prepares the test plan:** testers learn the system early and can identify test conditions and risks.

15.2 Sample Requirement Document: Online Shopping System (extract)

- **R1:** The system shall allow users to register using e-mail or mobile number.
- **R2:** The search page should load fast.
- **R3:** The user shall add products to the cart.

- **R4:** A user may cancel an order at any time.
- **R5:** Orders shall not be cancellable after they are shipped.
- **R6:** The system shall be user friendly and easy to use.
- **R7:** Discount shall be applied to eligible orders.
- **R8:** Payment can be made by credit card, debit card, UPI or net banking.
- **R9:** Delivery shall be done within a reasonable time.
- **R10:** The system shall handle a large number of users.
- **R11:** A refund shall be provided for returned products.
- **R12:** Passwords shall be secure.

15.3 Analysis of the Requirements

- **Ambiguous** - Requirement: R2 "load fast"; Problem: "Fast" is not defined; Suggested improvement: Search results shall be displayed within 2 seconds for 95% of requests with 5,000 concurrent users.
- **Ambiguous** - Requirement: R7 "eligible orders"; Problem: Eligibility is not defined; Suggested improvement: Define slabs/coupons, validity, maximum discount.
- **Ambiguous** - Requirement: R9 "reasonable time"; Problem: Not measurable; Suggested improvement: Delivery within 3-5 working days for metro cities.
- **Incomplete** - Requirement: R1 registration; Problem: No validation rules, mandatory fields, verification (OTP), or duplicate account handling; Suggested improvement: Add field rules, OTP verification and error messages.
- **Incomplete** - Requirement: R3 cart; Problem: No limit on quantity, stock check, price change, cart persistence; Suggested improvement: Define quantity limits, out-of-stock behaviour, cart retention period.
- **Incomplete** - Requirement: R8 payment; Problem: No failure handling, timeouts, refund to which mode, saved card rules; Suggested improvement: Add payment failure and retry flows.
- **Contradictory** - Requirement: R4 vs R5; Problem: R4 says cancel "at any time" while R5 says not after shipping; Suggested improvement: Cancellation allowed until the order is shipped.
- **Unverifiable** - Requirement: R6 "user friendly"; Problem: Cannot be proved by any test; Suggested improvement: A new user shall complete a purchase in at most 5 steps / 3 minutes in usability test with 90% success.
- **Unverifiable** - Requirement: R10 "large number of users"; Problem: No number given; Suggested improvement: Support 10,000 concurrent users with response under 3 seconds.
- **Unverifiable** - Requirement: R12 "secure"; Problem: Not measurable; Suggested improvement: Min 8 characters with complexity rules, stored hashed with salt, lock after 5 failed attempts.
- **Missing** - Requirement: Not mentioned anywhere; Problem: Return/exchange policy details, GST/tax calculation, address management, order tracking, wishlist, product reviews, notifications, admin functions, data privacy/consent, accessibility, supported browsers/devices, backup and recovery, audit logging, session timeout, stock management; Suggested improvement: Add functional and non-functional requirements for each.
- **Incomplete** - Requirement: R11 refund; Problem: Refund period, conditions, amount (shipping charges?), and time to credit are not given; Suggested improvement: Refund within 7 days of return approval to original payment mode.

15.4 Review Checklist for Requirements (before implementation)

- **1** - Checklist item: Is every requirement uniquely identified and traceable to a business need?; Check: [ ].
- **2** - Checklist item: Is each requirement clear, with only one possible interpretation (no words like fast, easy, reasonable, appropriate, etc.)?; Check: [ ].

- **3** - Checklist item: Is each requirement complete (inputs, processing, outputs, error cases, validation rules)?; Check: [ ].
- **4** - Checklist item: Are there any contradictions between requirements or with other documents?; Check: [ ].
- **5** - Checklist item: Is each requirement verifiable with measurable acceptance criteria?; Check: [ ].
- **6** - Checklist item: Are all user roles (guest, customer, seller, admin) and their permissions defined?; Check: [ ].
- **7** - Checklist item: Are boundary values, limits and data formats specified?; Check: [ ].
- **8** - Checklist item: Are exception and error handling requirements specified (payment failure, out-of-stock, network error)?; Check: [ ].
- **9** - Checklist item: Are non-functional requirements (performance, security, usability, reliability, compatibility, scalability) quantified?; Check: [ ].
- **10** - Checklist item: Are business rules (discount, tax, refund, cancellation) fully defined with examples?; Check: [ ].
- **11** - Checklist item: Are external interfaces defined (payment gateway, courier, SMS/e-mail services)?; Check: [ ].
- **12** - Checklist item: Are legal and regulatory requirements covered (data protection, GST, consumer rights)?; Check: [ ].
- **13** - Checklist item: Are requirements feasible within time, cost and technology constraints?; Check: [ ].
- **14** - Checklist item: Are requirements atomic (one requirement per statement) and free of design/implementation details?; Check: [ ].
- **15** - Checklist item: Is the terminology consistent and a glossary provided?; Check: [ ].
- **16** - Checklist item: Are priorities assigned (must / should / could)?; Check: [ ].
- **17** - Checklist item: Are assumptions, dependencies and constraints documented?; Check: [ ].
- **18** - Checklist item: Can test cases be derived for each requirement? Is a requirement-to-test traceability matrix possible?; Check: [ ].
- **19** - Checklist item: Are missing items (such as returns, notifications, audit, backup, accessibility) checked?; Check: [ ].
- **20** - Checklist item: Has the document been reviewed and approved by customer, developers and testers, with versioning and change control?; Check: [ ].

Conclusion: static black-box testing of the requirement document before implementation prevents ambiguity from becoming defects, reduces cost, and gives the testing team clear, testable requirements to start test design.